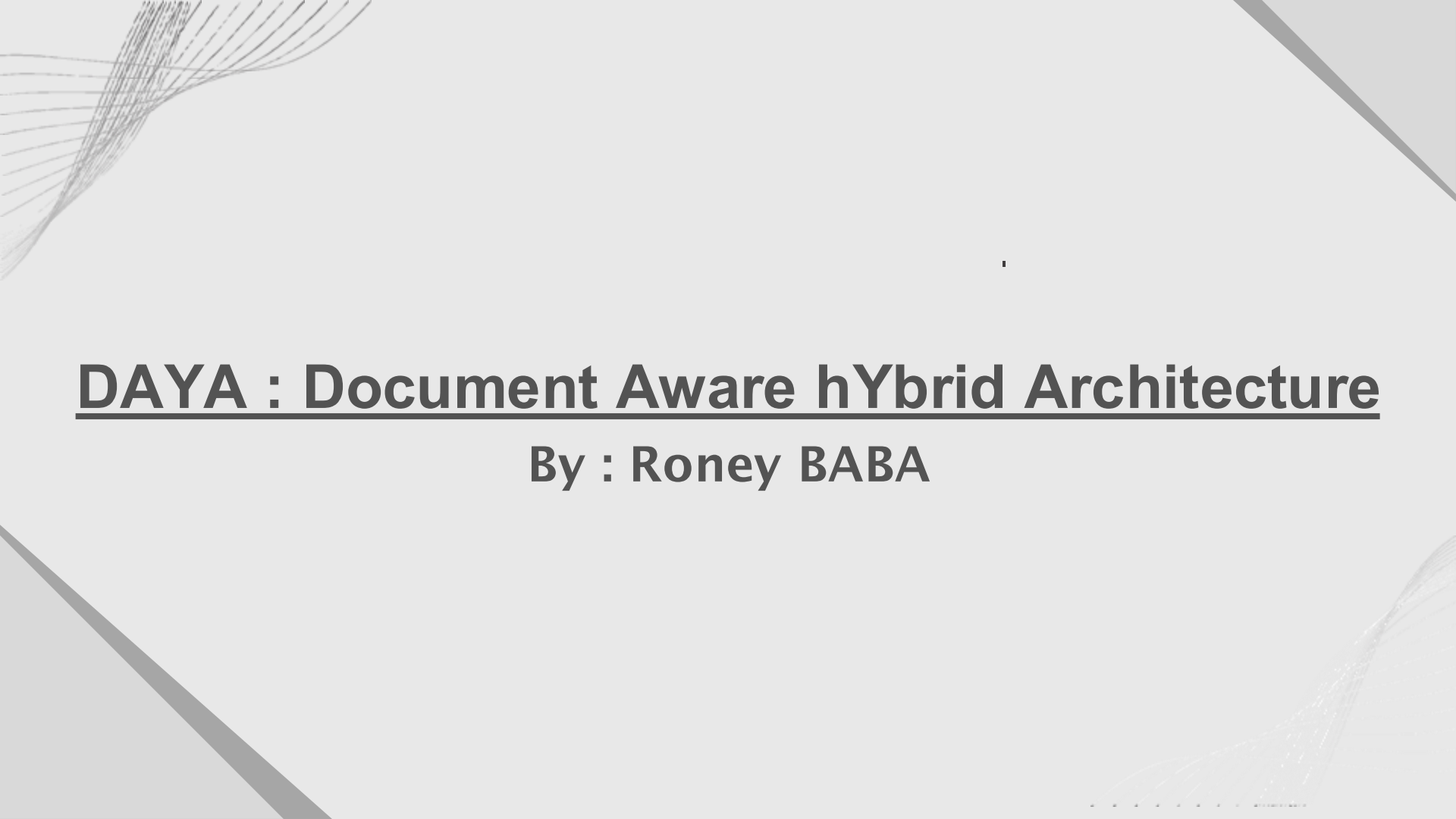
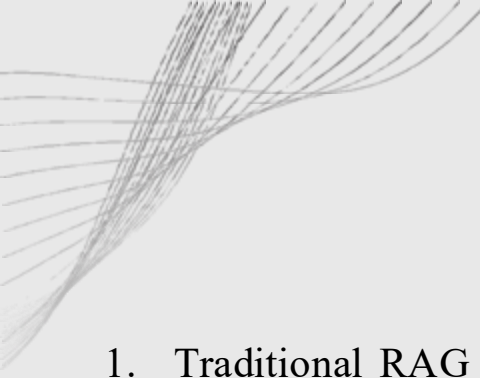


The background features a light gray field with abstract, thin, wavy lines in the top-left and bottom-right corners. A solid dark gray diagonal band runs from the top-right towards the bottom-left.

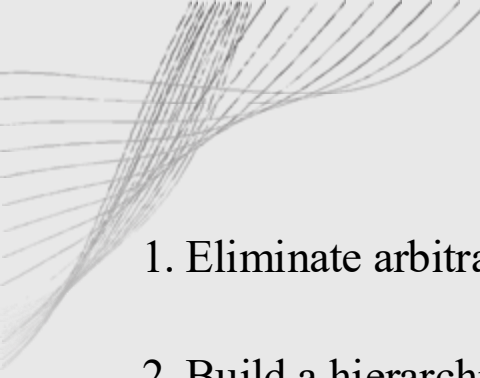
DAYA : Document Aware hYbrid Architecture

By : Roney BABA



Problem Statement

1. Traditional RAG pipelines rely on arbitrary token-based chunking and text-only OCR, causing loss of document structure and section hierarchy.
2. Current enterprise RAG pipelines lack dedicated frameworks for parsing corporate presentations. As a result, critical visual insights from charts and graphs are entirely lost when processing long, highly sensitive slide decks.

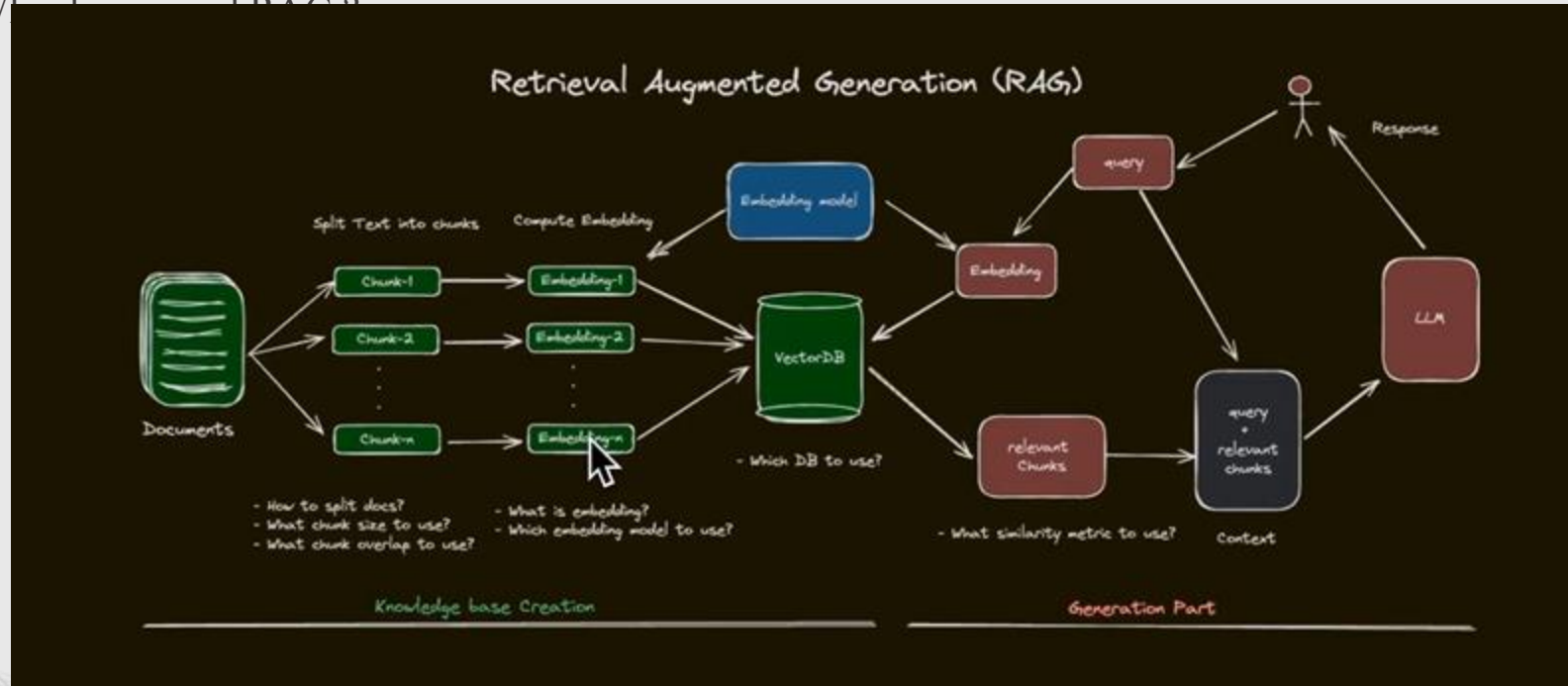


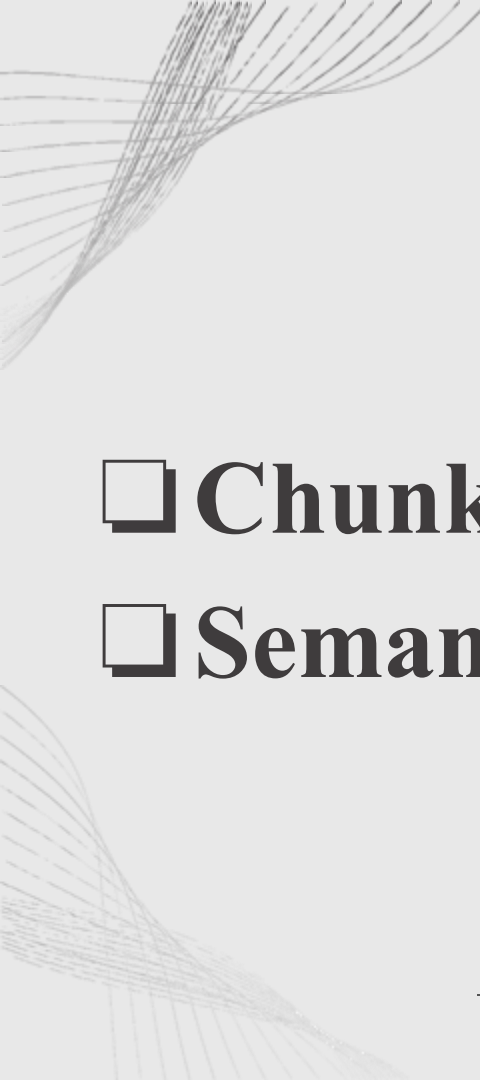
Objectives

1. Eliminate arbitrary token chunking
2. Build a hierarchical document index in form of a tree
3. Integrate multi-modal processing
4. Enable intelligent query handling and retrieval
5. Deliver grounded and traceable responses
6. Design a modular, production-ready pipeline

Introduction

- What is RAG?





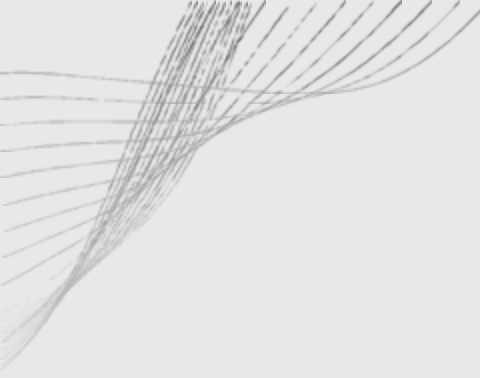
Issues

❑ **Chunking**

❑ **Semantic Search**



Introducing DAYA



Why DAYA Works?

- ❑ **NO Chunking**
- ❑ **Relevant searching**

Loss of Context

The Question: *"Where is the inspection checklist before lifting"*

What a Complete Answer Requires:

1. The 5-point inspection checklist on Page 20 (*Ropes, Chains, Hooks, Safety Latch, Drum Spooling*)

Traditional RAG

Returned responsibility bullets from Page 19 instead of the actual checklist on Page 20.

Why it failed — Chunk Isolation:

- Content is split into fixed chunks processed independently
- The retrieved chunk ended exactly at *"At a minimum, the installer must..."*
- The actual checklist items lived in the **next chunk** — never seen by the model
- No mechanism to look across chunk boundaries for continuation of content
- Result: A confident-sounding but factually wrong answer pointing to the wrong list

How DAYA Solves

Correctly identified the inspection checklist on Page 20 and reproduced all 5 items accurately. (*Ropes, Chains, Hooks, Safety Latch, Drum Spooling*)

Why it succeeded — No Chunking:

- Operates at the document-structure level: *section* → *page* → *figures*
- The checklist header and its items were never split across boundaries
- Kept the checklist content and associated rigging instructions together as one coherent unit
- No artificial chunk cutoffs meant no content was left just out of reach
- Result: The complete, correct list — retrieved and reproduced in full

Lack of Reasoning

The Question: *"How do I change the dehydrator and what safety precautions apply?"*

What a Complete Answer Requires:

1. Dehydrator replacement procedure — Page 29
2. Refrigerant-related safety warnings — Page 6

Traditional

RAG

Retrieved the mechanical procedure on Page 29, but missed the critical Refrigerant Warning on Page 6.

Why it failed — Static, Single-Hop Retrieval:

- Matched content based on direct keyword similarity only
- Retrieved the dehydrator procedure and stopped there
- Could not reason that a dehydrator works with refrigerant
- Never made the second "hop" to connect the procedure → working fluid → refrigerant warning
- Result: A partial answer that omits a critical safety hazard

How DAYA Solves

Retrieved both the dehydrator replacement procedure (Page 29) AND the critical Refrigerant Warning (Page 6).

Why it succeeded — Structure-Aware Reasoning:

- Located the dehydrator replacement procedure on Page 29
- Noticed Step 4 referenced "*refrigerant leaks*" — flagged it as a logical gap
- Inferred that refrigerant safety guidance was relevant to complete the answer
- Followed document structure to the safety section and retrieved the Refrigerant Warning on Page 6
- Result: A complete, safe, and accurate answer — no critical hazard left behind

Hallucinations

The Question: *"When is glycol pumping required in closed-loop free cooling? "*

What a Complete Answer Requires:

1. An accurate reflection of the Closed-Loop configuration as described on Pages 20–21 — without mixing in open-loop details.

Traditional

RAG

Retrieved the mechanical procedure on Page 29, but missed the critical Refrigerant Warning on Page 6.

Why it failed — Context Confusions:

- Both open-loop and closed-loop sections share overlapping terms: *"free cooling," "valve," "glycol," "flow"*
- Vector RAG pulled passages from **both configurations** simultaneously
- The LLM, now seeing mixed context, merged details across systems
- Gaps were filled with plausible-sounding guesses rather than verified facts
- Result: A hallucinated answer that contradicts the manual — wrong system, wrong pump role

How DAYA Solves

Accurately answered using only the Closed-Loop section (Pages 20–21), without importing any open-loop details.

Why it succeeded — Structure-Aware Reasoning:

- Used document structure to navigate directly to the Closed-Loop section
- Naturally excluded the Open-Loop content that shared overlapping terminology
- Never conflated the free-cooling valve (open-loop only) with closed-loop components
- Connected only the logical points relevant to the closed-loop glycol pump
- Result: A clean, accurate answer with zero cross-contamination from adjacent sections

Tech Stack Used

Category	Technologies & Tools Used
Language & Core	Python 3.10+, Python Abstract Base Classes (ABCs), Google Colab SDK
LLMs & AI Models	openai/gpt-oss-120b , Meta Llama 4 Scout (llama-4-scout-17b-16e-instruct)
Document Parsing	IBM Docling , PyPDF
Embeddings	Jina AI Embeddings v5 (jina-embeddings-v5-text-small) , SentenceTransformers
Vector Database	ChromaDB (In-memory & Persistent Client)
Inference & Cloud APIs	Groq Cloud API

System Flow Chart





System Architecture

1. Document Input

Accepts multi-format files — PDF, PPTX, and DOCX — as raw input to the pipeline.

2. Layout Engine

IBM Docling parses reading order, structural headers, and tables. Images are detected as `PictureItem` objects and routed to Meta Llama 4 Scout (VLM), which converts them into structured text descriptions.

3. Tree Builder

Document sections and headers are mapped into a nested JSON hierarchy. Each child node automatically inherits contextual metadata — page range, position index, and parent heading — from its parent node.

4. Embed & Store

Every node's text is embedded using Jina AI Embeddings v5 (supports up to 8192 tokens). The resulting vectors along with their metadata are upserted into ChromaDB for persistent storage.

System Architecture

5. Retrieval Engine

The incoming user query is decomposed into individual sub-questions by gpt-oss-120b running on Groq. Each sub-question is independently embedded and matched against ChromaDB using a cosine distance threshold filter. Surviving nodes are ranked by a frequency-weighted engine to identify the most relevant sections.

6. LLM Inference & Response

Top-ranked nodes are assembled in reading order and injected into the LLM's context window. Groq (gpt-oss-120b) generates the final answer, strictly grounded in retrieved content, with page-level citations appended to every statement.



Outcomes

1. Context-intact document retrieval
2. Zero visual data loss
3. Hallucination-free responses
4. Scalable enterprise-grade architecture
5. Modular and swappable codebase



THANK YOU